

Laya Extension to the Semantic Cache

A simple technical summary for the FSCgRPC team

Purpose

The original system reduces repeated LLM inference with an exact cache and semantic vector search. Laya was added as a decision layer that evaluates whether a retrieved cached response is safe to reuse. The target trade-off is semantic recall, precision, and latency.

Original Architecture

1. A client sends a gRPC request to the Go gateway.
2. Redis checks an exact SHA-256 cache key.
3. On an exact miss, the Python encoder creates a 384-dimensional embedding.
4. Redis RediSearch performs HNSW nearest-neighbor search.
5. Shannon entropy selects a query-specific distance threshold.
6. A valid cache result is returned; otherwise the request falls back to Ollama generation.

Original paper baseline: 901 queries, 47.4% combined cache hit rate, and an 85% latency reduction. Those numbers describe the pre-Laya experiment.

Laya Extension

The Python encoder exposes `GetEmbedding` and `VerifyCachedResponse`. Laya receives the new query, the nearest cached query, and the cached response. It returns a probability interpreted as the probability that the cached response is safe to return.

```
Exact Redis lookup
| hit -> return exact response
| miss
v
Embedding + HNSW search
v
Nearest cached pair = candidate
v
Shannon signal + candidate band
v
Laya verification
+ approved -> cached response
+ rejected -> new generation
```

What is a candidate?

A candidate is the nearest cached query-response pair returned by Redis before final acceptance. It is not automatically a cache hit. It may be a valid paraphrase or a dangerous near-match such as London versus Tokyo, hello versus goodbye, Python versus Golang, or MySQL versus PostgreSQL.

Policy Modes

`behind_shannon`: Laya is called only when Shannon accepts. `candidate_band`: Laya can also inspect candidates within a broader distance band. The current default settings are a candidate distance of 0.35 and a Laya approval threshold of 0.80.

Telemetry and Storage

Standard request telemetry is published to cache-telemetry for the dashboard. Detailed Laya audit events are published to laya-cache-audit. The audit records the distance, Shannon result, candidate eligibility, Laya probability and decision, Laya latency, total latency, and final response.

300-Request Results

Metric	Result
Total requests	300
Exact hits	75
Laya-approved semantic hits	25
Total cache hits	100 (33.3%)
Laya calls	141
Laya approvals / rejections	25 / 116
Laya approval rate	17.7%
Average Laya latency	1,008 ms
Average total latency	2,607 ms

The broader candidate band exposed 74 candidates to Laya that Shannon alone would have rejected.

What Worked

Laya approved useful paraphrases involving soccer rules, Moon distance, the first US president, freezing point, speed of light, CSV parsing, Go HTTP requests, and flu symptoms. This confirms that Laya can recover some matches beyond a strict Shannon gate.

Problems Found

False positives: a goodbye query received a hello response in Spanish, and a London population query received a Tokyo response. Both crossed the 0.80 threshold. This means Laya can overlook critical entity or intent tokens.

False negatives: many likely valid paraphrases were rejected just below 0.80, including France capital, chocolate cake, Harry Potter, Mona Lisa, coffee, REST API, and C++ queue queries.

Latency: Laya verification averaged about one second. Rejected candidates averaged about 3.9 seconds total because the request paid for Laya and then paid for fallback generation.

Technical Decisions

- Redis exact lookup and HNSW retrieval remain the cache foundation.
- Shannon remains an independent deterministic signal.
- Laya is a semantic safety verifier, not the retriever.
- A broader candidate band gives Laya a chance to recover valid paraphrases.
- Independent Shannon, candidate, Laya, and final decisions are logged.
- The protobuf contract remains unchanged.
- Laya audit data uses a separate Kafka topic.

Next Work

1. Add critical-token checks for entities, numbers, dates, languages, databases, frameworks, negation, and time words.
2. Calibrate Laya on labelled cache-equivalence examples.
3. Tune candidate and approval thresholds on held-out data.
4. Return exact hits before waiting for embedding and model-threshold work.
5. Remove the unused online Model AST request from the critical path.
6. Measure warm/cold, P50, and P95 latency.
7. Compare Shannon-only, Shannon-gated Laya, and candidate-band Laya on identical cache state.
8. Fine-tune Laya on paraphrases and negative entity/constraint substitutions.

Bottom Line

The extension successfully adds a second semantic decision maker and better observability, but the current run does not prove an overall performance improvement. Laya improves some semantic recall, adds about one second of verification cost, and still permits critical entity substitutions. The next target is a layered policy: exact cache, broad retrieval, critical-token checks, calibrated Laya verification, and generation fallback.